Inés Larrañaga Fernández de Pinedo	Egibide
	SGE

Table of Contents

Layout
- Template tag {% url %}

Paginación
- Modificación de `listar_empleados`

Subida de ficheros
- Modificación del modelo `Empleado` para subir una imagen
- Configurar los archivos estáticos y medios en `settings.py`
- Configurar las URLs para servir archivos en Desarrollo
- Crear un formulario para subir la imagen
- Modificar la vista para manejar la subida de archivos
- Mostrar la imagen en el detalle de empleado

Layout

En el desarrollo web, un **layout** es la estructura base de una página, definiendo la organización de elementos como encabezado, barra de navegación, contenido y pie de página. En los templates que se han creado hasta ahora, en todas ellas no estamos añadiendo ningún layout común, lo cual suele ser habitual. ¿Porqué? En cada template podríamos añadir siempre el código html, css, y js del layout, pero estaríamos repitiendo código en todas los templates.

Así que vamos a crear un layout muy simple, pero que os sirva de referencia para cuando hagáis el vuestro con Bootstrap y saber así cómo hacer que ese código se comparta en todos los templates que generemos.

Para ello, Django proporciona un sistema de herencia de plantillas que es lo que vamos a abordar a continuación. Vamos a crear el fichero `layout.html` dentro de `templates`. Podemos escribir html:5 y presionar tabulador para generar una plantilla HTML bien estructurada (podemos poner lang=es y el título):

```html
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="ie=edge">
    <title>Mi CRM</title>
</head>
<body>
    <h1>Mi CRM</h1>
    <ul>
        <li><a href="/">Portada</a></li>
        <li><a href="/empresa/empleados/">Empleados</a></li>
        <li><a href="/">Departamentos</a></li>
        <li><a href="/">Habilidades</a></li>
    </ul>
    {% block content %}{% endblock %}
</body>
</html>
```

Dentro del html verás como hay código que no es propiamente HTML, como `{% block content %}{% endblock %}`. Esto es un *template tag* y sirve para añadir lógica de programación dentro del propio HTML. Este `template tag` pertenece al `DTL - DJango Template Language` que es el lenguaje que entiende el motor de plantillas de Django para renderizar las vistas. De forma general hay 3 tipos:

- **Variables**: cuando el motor de plantillas encuentra una variable la reemplaza por su valor. Ejemplos:

  ```
  {{ variable }} {{object.attribute}}
  ```

- **Filtros**: permite modificar la variable a reemplazar (mostrar en minúsculas, mostrar las primeras X palabras,...). Ejemplos:

  ```
  {{ name|lower }} {{ bio|truncatewords:30 }}
  ```

- **Etiquetas (tags)**: más complejos que las variables, algunas generan texto de salida, ejecutan una lógica, cargan otra plantilla, etc.

Os dejo un enlace a la documentación oficial aquí: Templates | Django documentation | Django

En este caso el template tag **block** sirve para definir un bloque de contenido con un nombre. En este caso su nombre es `content`. Ese contenido se irá cambiando por el contenido del template en cuestión, manteniendo el layout definido en un único punto.

Si probamos ahora la aplicación veréis como el menú y el título no aparecen, porque no hemos hecho referencia al `layout.html` desde ningún sitio. Para solucionarlo a cada una de los templates tenemos que decirles que son hijas de la plantilla `layout.html`. Por ejemplo, cambia el template `empresa/listar_empleados.html` de la siguiente forma:

```
{% extends 'layout.html' %}
{% block content %}
<h1>Lista de Empleados</h1>
<table border="1">
    <thead>
    <tr>
        <th>Nombre</th>
        <th>Fecha de Nacimiento</th>
        <th>Antigüedad</th>
        <th>Departamento</th>
        <th>Habilidades</th>
        <th>Acciones</th>
    </tr>
    </thead>
    <tbody>
    {% for empleado in empleados %}
        <tr>
            <td><a href="{% url 'ver_empleado' empleado.id %}">{{
empleado.nombre }}</a></td>
            <td>{{ empleado.fecha_nacimiento }}</td>
            <td>{{ empleado.antiguedad }}</td>
            <td>{{ empleado.departamento.nombre }}</td>
            <td>{% for habilidad in empleado.habilidades.all %}{{
habilidad.nombre }}{% if not forloop.last %},
            {% endif %}{% endfor %}</td>
            <td>
                <a href="{% url 'editar_empleado' empleado.id %}">Editar</a>
                <a href="{% url 'eliminar_empleado' empleado.id
%}">Eliminar</a>
            </td>
        </tr>
    {% endfor %}
    </tbody>
</table>
<div class="pagination">
    <span class="step-links">
        {% if empleados.has_previous %}
            <a href="?page=1">« Primera</a>
            <a href="?page={{ empleados.previous_page_number }}">Anterior</a>
        {% endif %}

        <span class="current">
            Página {{ empleados.number }} de {{ empleados.paginator.num_pages
}}.
        </span>

        {% if empleados.has_next %}
```

```
            <a href="?page={{ empleados.next_page_number }}">Siguiente</a>
            <a href="?page={{ empleados.paginator.num_pages }}">Última »</a>
        {% endif %}
    </span>
</div>
<a href="{% url 'crear_empleado' %}">Crear Nuevo Empleado</a>
{% endblock %}
```

Si probamos ahora veremos que el layout y el contenido se renderizan correctamente.

> Por tu cuenta, modifica todos los templates para que entiendan del `layout.html` definido.

Template tag {% url %}

Vamos a mencionar un template tag muy útil, que nos permite hacer referencia a cualquier view desde cualquier template sin miedo a equivocarnos.

Para ello, abre primero el fichero `empresa/urls.py`, y comprueba que todas las urls tengan un `name` definido:

```
urlpatterns = [
    path('empleados/', listar_empleados, name='listar_empleados'),
    path('empleados/<int:empleado_id>', ver_empleado, name='ver_empleado'),
    path('empleados/crear/', user_passes_test(es_gerente)(crear_empleado),
name='crear_empleado'),
    path('empleados/editar/<int:empleado_id>/', editar_empleado,
name='editar_empleado'),
    path('empleados/eliminar/<int:empleado_id>/', eliminar_empleado,
name='eliminar_empleado'),
]
```

Esos nombres son los que utilizaremos en cualquier parte de la aplicación para genera un enlace a esa ruta o vista asociada.

De forma que podemos cambiar por ejemplo el layout como sigue:

```html
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="ie=edge">
    <title>Mi CRM</title>
</head>
<body>
    <h1>Mi CRM</h1>
    <ul>
        <li><a href="/">Portada</a></li>
        <li><a href="{% url 'listar_empleados' %}">Empleados</a></li>
        <li><a href="/">Departamentos</a></li>
        <li><a href="/">Habilidades</a></li>
    </ul>
    {% block content %}{% endblock %}
</body>
```

> Por tu cuenta haz lo mismo con todos los enlaces que aparecen en el template `empresa/listar_empleados.html`.

Paginación

La **paginación** es el proceso de dividir una lista de datos en páginas para que no tengas que mostrar todos los elementos en una sola vista. Esto es útil cuando trabajas con grandes cantidades de datos (como una lista de artículos, productos, o comentarios) para mejorar la experiencia del usuario y evitar que la página se vuelva demasiado lenta.

La paginación es muy útil para manejar grandes cantidades de datos sin sobrecargar la interfaz. En Django, se puede hacer de manera sencilla utilizando el objeto `Paginator`.

Modificación de `listar_empleados`

Dentro del fichero `empresa/views.py` vamos a importar el módulo de paginación:

```python
from django.core.paginator import Paginator
from django.shortcuts import render
```

Vamos a modificar el método `listar_empleados` para que una vez se obtengan los objetos que quieres mostrar, se pasen al `Paginator` para dividirlos en páginas:

```
def listar_empleados(request):
    empleados = Empleado.objects.all()
    # Configurar la paginación
    paginator = Paginator(empleados, 10)  # Muestra 10 empleados por página

    # Obtener el número de página desde la solicitud GET
    pagina = request.GET.get('page',1)  # Si no hay parámetro 'page', usar 1
(primera página)

    # Obtener los empleados para la página solicitada
    empleados_paginados = paginator.get_page(pagina)

    # Pasar los empleados paginados al template
    return render(request, 'empresa/listar_empleados.html', {'empleados':
empleados_paginados})
```

Así lo único que faltaría sería iterar sobre los empleados paginados que se han pasado al template y mostrar los controles de paginación.

```
...
</table>
<div class="pagination">
    <span class="step-links">
        {% if empleados.has_previous %}
            <a href="?page=1">« Primera</a>
            <a href="?page={{ empleados.previous_page_number }}">Anterior</a>
        {% endif %}

        <span class="current">
            Página {{ empleados.number }} de {{ empleados.paginator.num_pages
}}.
        </span>

        {% if empleados.has_next %}
            <a href="?page={{ empleados.next_page_number }}">Siguiente</a>
            <a href="?page={{ empleados.paginator.num_pages }}">Última »</a>
        {% endif %}
    </span>
</div>
```

Ahora el objeto que le pasamos al template tiene los siguientes atributos que ayudan a la implementación de los controles de la paginación:

- **`has_next`** : Retorna `True` si hay una página siguiente.

- `previous_page_number` : Número de la página anterior.
- `next_page_number` : Número de la página siguiente.
- `number` : El número de la página actual.
- `paginator.num_pages` : El número total de páginas.

Subida de ficheros

La **subida de archivos** es una funcionalidad común en muchas aplicaciones web, que permite a los usuarios cargar y almacenar archivos en el servidor o en servicios de almacenamiento en la nube. En **Django**, esta funcionalidad se maneja de manera sencilla mediante el uso del campo `FileField` o `ImageField` en los modelos, y con un formulario que permite la interacción del usuario.

Modificación del modelo `Empleado` para subir una imagen

En el modelo `Empleado`, vamos a agregar un nuevo campo de tipo `ImageField` que almacenará la foto del empleado. Este campo permitirá que se suba una imagen desde el formulario y se almacene en una carpeta dentro de la estructura de archivos del proyecto.

Aquí está la modificación en `empresa/models.py`:

```
class Empleado(models.Model):
    departamento = models.ForeignKey(Departamento, on_delete=models.CASCADE)
    habilidades = models.ManyToManyField(Habilidad)
    nombre = models.CharField(max_length=40)
    fecha_nacimiento = models.DateField()
    antiguedad = models.IntegerField(default=0)
    created = models.DateTimeField(auto_now_add=True,
                                   verbose_name="Fecha de creación")
    updated = models.DateTimeField(auto_now=True,
                                   verbose_name="Fecha de edición")

    # Nuevo campo para la imagen del empleado
    imagen = models.ImageField(upload_to='empleados/', null=True, blank=True)

    def __str__(self):
        return f"{self.nombre} --- {self.antiguedad}"
```

- `upload_to='empleados/'` : Este argumento define la carpeta donde se guardarán las imágenes de los empleados. En este caso, las imágenes se guardarán dentro de la carpeta `media/empleados/`.
    - Si no defines `upload_to`, las imágenes se guardarían en el directorio raíz de `media/` con un nombre generado aleatoriamente. Puesto que en un mismo proyecto puede

ocurrir que distintas entidades tengan ficheros asociados, se suele indicar el nombre de la entidad como carpeta contenedora de esas imágenes.

- `null=True`: Esto permite que el campo `imagen` sea opcional en la base de datos (si el empleado no tiene imagen).
- `blank=True`: Permite que el campo sea opcional en los formularios (esto es útil si no se requiere que el empleado tenga una imagen cuando se crea o actualiza).

Para poder usar ImageField o FileField hay que instalar el paquete:

```
pip install pillow
```

Configurar los archivos estáticos y medios en `settings.py`

Django va a necesitar saber cómo manejar los archivos subidos. Para ello, necesitas configurar el manejo de archivos estáticos y medios en el archivo `settings.py`.

```
import os

# Configuración para los archivos subidos
MEDIA_URL = '/media/'  # URL para acceder a los archivos subidos
MEDIA_ROOT = os.path.join(BASE_DIR, 'media')  # Ruta donde se almacenarán los
archivos subidos
```

Estas configuraciones definen cómo Django manejará los archivos subidos por los usuarios.

Configurar las URLs para servir archivos en Desarrollo

Cuando trabajas en un entorno de desarrollo, es necesario asegurarte de que Django sirva los archivos subidos correctamente. Para ello, en tu archivo `urls.py`, necesitas agregar el siguiente código al final:

```
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    # Otras rutas
] + static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

Crear un formulario para subir la imagen

Ahora que tenemos el campo `imagen` en el modelo, vamos a crear un [formulario](Working with forms | Django documentation | Django) para que los usuarios puedan subir una imagen de un empleado. Utilizaremos un formulario basado en `ModelForm` para facilitar la validación y la creación del formulario. Así que modificaremos sustancialmente el contenido de la función `crear_empleado`. Para ello crea un fichero `forms.py` con el siguiente contenido:

```
from django import forms
from .models import Empleado

class EmpleadoFormulario(forms.ModelForm):
    class Meta:
        model = Empleado
        fields = ['departamento', 'habilidades', 'nombre', 'fecha_nacimiento',
'antiguedad', 'imagen']
```

Este formulario generará automáticamente los campos que definimos en el modelo, incluyendo el campo de la imagen.

Modificar la vista para manejar la subida de archivos

Ahora vamos a modificar la vista que maneja la creación de empleados, para que también maneje la subida de imágenes. Es importante incluir el parámetro `enctype="multipart/form-data"` en el formulario HTML para permitir la carga de archivos.

```
def crear_empleado(request):
    if request.method == 'POST' and request.FILES:
        form = EmpleadoFormulario(request.POST, request.FILES)
        if form.is_valid():
            form.save()  # Guarda el nuevo empleado con la imagen
            return redirect('listar_empleados')  # Redirige al listado en caso
de
    else:
        form = EmpleadoFormulario()  # Si no es un POST, muestra el formulario
vacío

    departamentos = Departamento.objects.all()
    habilidades = Habilidad.objects.all()
    return render(request, 'empresa/crear_empleado.html', {
        'form': form,
        'departamentos': departamentos,
        'habilidades': habilidades,
    })
```

De forma que el template `empresa/crear_empleados.html` quedaría como sigue:

```
<body>
    <h1>Crear Empleado</h1>
    <form method="POST" enctype="multipart/form-data">
        {% csrf_token %}
        {{ form.as_p }}
        <button type="submit">Crear</button>
    </form>
    {% if form.errors %}
        <div class="errors">
            <p>Hubo errores en el formulario:</p>
            <ul>
                {% for error in form.errors %}
                    <li>{{ error }}</li>
                {% endfor %}
            </ul>
        </div>
    {% endif %}
</body>
```

- **`enctype="multipart/form-data"`** : Este es el atributo clave para que el formulario pueda enviar archivos. Es indispensable para el correcto envío vía HTTP del fichero.
- **`{{ form.as_p }}`** : Django se encarga de renderizar automáticamente los campos del formulario, incluidas las validaciones, los errores, y el campo para la imagen. Los campos serán renderizados en un formato simple (por párrafo).
- **`{% csrf_token %}`** : El token CSRF es necesario para la seguridad del formulario. Lo genera el backend cuando crea la vista y la devuelve en la primera petición GET. Este queda oculto en el formulario y en el POST se chequea en el servidor.

Realiza pruebas convenientemente para ver que el registro de empleados funciona, y en caso de error se muestran los errores.

Mostrar la imagen en el detalle de empleado

Finalmente, para mostrar la imagen del empleado en el template de `ver_empleado.html` :

```
<table border="1">
        <thead>
            <tr>
                <th>Imagen</th>
                <th>Nombre</th>
                <th>Fecha de Nacimiento</th>
                <th>Antigüedad</th>
                <th>Departamento</th>
                <th>Habilidades</th>
            </tr>
        </thead>
        <tbody>
            <tr>
                <td>
                    {% if empleado.imagen %}
                        <img style="max-width: 10em; height: auto; object-fit:
cover;" src="{{ empleado.imagen.url }}" alt="Imagen de {{ empleado.nombre }}"
/>
                    {% else %}
                        <p>No hay imagen disponible.</p>
                    {% endif %}
                </td>
                <td>{{ empleado.nombre }}</td>
                <td>{{ empleado.fecha_nacimiento }}</td>
                <td>{{ empleado.antiguedad }}</td>
                <td>{{ empleado.departamento.nombre }}</td>
                <td>{% for habilidad in empleado.habilidades.all %}{{
habilidad.nombre }}{% if not forloop.last %}, {% endif %}{% endfor %}</td>
            </tr>
        </tbody>
    </table>
```

- `empleado.imagen.url` devuelve una cadena que representa la ruta donde la imagen está almacenada, para que el navegador pueda acceder a esa imagen y mostrarla en la página web.

> **Pregunta1**: ¿Cómo se puede restringir el tamaño o tipo de ficheros que se pueden subir? Consulta
>
> **Pregunta2**: Suele ser interesante renombrar los archivos subidos por si hubiera dos usuarios o el mismo que subieran el fichero con el mismo nombre. ¿Puedes cambiar el código para que se renombre el archivo y la ruta guardada en BD?